

HW1 Experiments

Clarified Instructions and Reporting

Experiment 1 Overview

- Purpose: Measure write performance with different block sizes
- Create 18 files named 01, 02, ..., 18
- Allocate 256 KB memory (malloc/new)
- Fill memory with cyclic values 1–255

Code Sample

```
size_t size = 256 * 1024; // 256 KB
unsigned char *buffer = (unsigned char *)malloc(size);

if (buffer == NULL) {
    perror("malloc failed");
    return 1;
}

// 將記憶體填入 1~255 循環
for (size_t i = 0; i < size; i++) {
    buffer[i] = (i % 255) + 1;
    // 這樣會依序填 1, 2, ..., 255，再循環回 1
}
```

Experiment 1 Detailed Steps

1. Create files 01 to 18
2. Allocate and fill memory
3. One phase For each file n:
 - Use write() system call
 - Block size = 2^n bytes (based on filename)
 - loop while all data in memory are written
4. Run 256 phases
5. Record execution time for each writing phase

Experiment 1 Notes

- File 01: $2^0 = 1$ byte
- File 18: $2^{17} = 131,072$ bytes
- Goal: Observe how block size impacts performance
- Ensure timing is accurate (start before write, stop after finish)

Experiment 2 Overview

- Purpose: Compare sequential and random read performance
- Create file 19
- Allocate 256 KB memory and fill with values 1–255

Experiment 2 Detailed Steps

1. Create file 19
2. Write memory to file $2^{10} = 1024$ times, close.

Test 1: Open 19 and ..

Sequential read: read entire file from start to end

Test 2: Open 19 and ..

Random read: perform 1024 reads of 256 KB from random offsets

Record execution time

Experiment 2 Notes

- File must be large enough for 1024 reads
- Use `lseek()` to move file pointer for random access
- Compare sequential vs random read performance

Report Requirements

- Prepare a PowerPoint report
- Include charts for Experiment 1 and 2
 - - X-axis: block size or read type
 - - Y-axis: execution time (ms)
- Provide analysis:
 - - Why larger block sizes may be faster
 - - Compare sequential and random reads
 - - Conclusions and insights

Example Chart

- Experiment 1: Write time vs block size (bar/line chart)
- Experiment 2: Sequential vs Random read times
- Highlight performance differences and trends

Conclusion Guidelines

- Summarize findings of both experiments
- Practical implications: system design considerations
- Sequential vs random access: performance impact
- Lessons learned

Non-buffered File I/O in C

Relevant system calls for HW1
experiments

Overview

- Non-buffered I/O: direct system calls (no stdio buffering)
- Functions are lower-level compared to `fopen/fread/fwrite`
- Useful for precise control in performance experiments
- Related to HW1 tasks: file creation, write, read, random access

open()

- Header: `#include <fcntl.h>`
- Purpose: Open or create a file
- Example:

```
int fd = open("file.dat", O_WRONLY |  
O_CREAT, 0644);
```
- Returns file descriptor (fd) or -1 on error

write()

- Header: `#include <unistd.h>`
- Purpose: Write data to a file
- Example:
 - `ssize_t bytes = write(fd, buffer, length);`
- Parameters:
 - `fd` - file descriptor
 - `buffer` - pointer to data
 - `length` - number of bytes to write

read()

- Header: `#include <unistd.h>`
- Purpose: Read data from a file
- Example:
 - `ssize_t bytes = read(fd, buffer, length);`
- Used in HW1 Experiment 2 for sequential and random reads

lseek()

- Header: `#include <unistd.h>`
- Purpose: Move file offset (for random access)
- Example:
 - `off_t pos = lseek(fd, offset, SEEK_SET);`
- In HW1: move to random offset before `read()`

close()

- Header: `#include <unistd.h>`
- Purpose: Close an opened file
- Example:
- `close(fd);`
- Always close files after operations to free resources

Summary

- Non-buffered I/O calls in C:
- `open()`, `write()`, `read()`, `lseek()`, `close()`
- Provide low-level, direct control over file operations
- Essential for HW1 experiments to measure raw performance
- Different from buffered stdio functions (`fopen`, `fread`, `fwrite`)